Chapter 6: Pointers and Modular Programming

Problem Solving & Program Design in C

Seventh Edition

By Jeri R. Hanly &
Elliot B. Koffman

Introduction

- Pointers
  - Powerful, but difficult to master
  - Simulate call-by-reference
  - Close relationship with arrays and strings

- Pointers are usually difficult for new C programmers to comprehend and use. If you remember the following simple statement, working with pointers should be less painful…

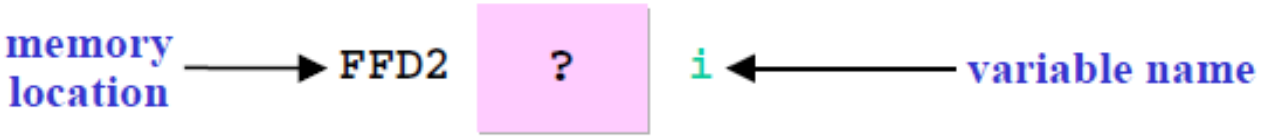
POINTERS CONTAIN MEMORY ADDRESSES, NOT DATA VALUES!

Pointer Variable Definitions and Initialization

- When you declare a simple variable, like

```
int i;
```

- a memory location with a certain address is set aside for any values that will be placed in i. We thus have the following picture:



- After the statement `i=35;` the location corresponding to `i` will be filled



7.2 Pointer Variable Definitions

- You can find out the **memory address of a variable** by simply using the address operator `&`. Here is an example of its use:

```
&v
```

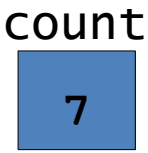
- The above expression should be read as "address of `v`", and it returns the memory address of the variable `v`.

- The following simple program demonstrates the difference between the contents of a variable and its memory address:

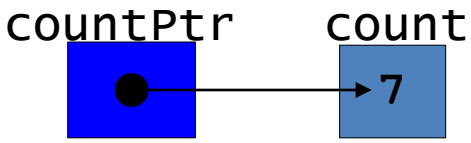
```
#include <stdio.h>
int main(void){
float x;
x =5;
printf("%p",&x); // output is memory address like 0028FF1C
return 0;
}
```

Pointer Variable Definitions and Initialization

- Pointer variables
  - Contain memory addresses as their values
  - Normal variables contain a specific value (direct reference)



  - Pointers contain address of a variable that has a specific value (indirect reference)
  - Indirection – referencing a pointer value



Pointer Variable Definitions and Initialization

- Pointer definitions
  - ***** used with pointer variables

```
int *myPtr;
```

  - Defines a pointer to an `int` (pointer of type `int *`)
  - Multiple pointers require using a `*` before each variable definition

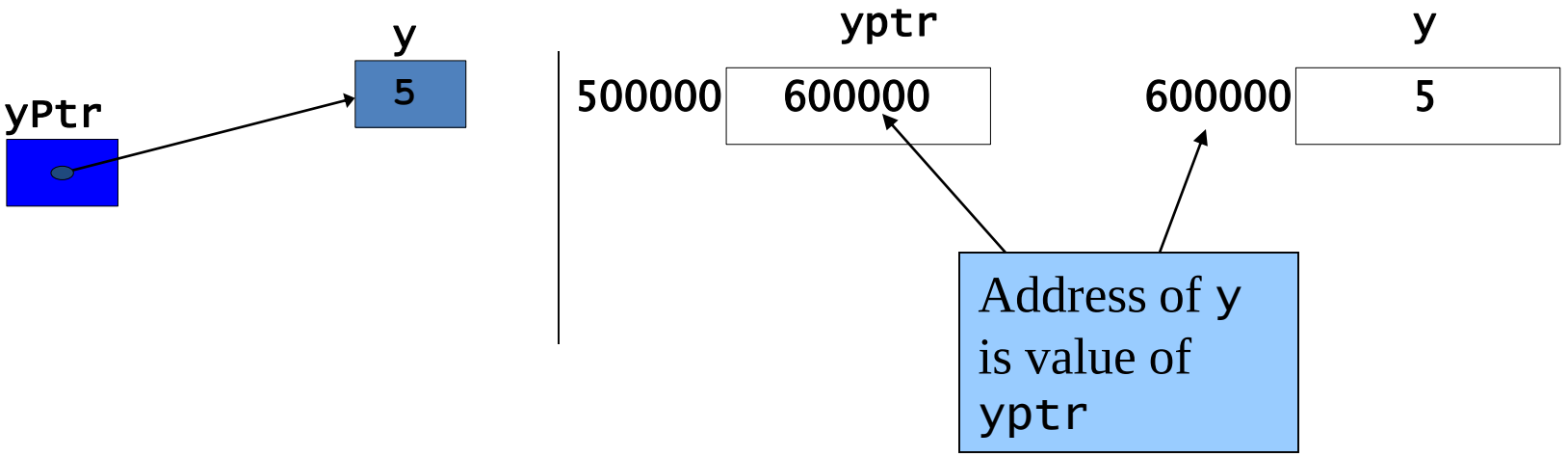
```
int *myPtr1, *myPtr2;
```

  - Can define pointers to any data type

Pointer Operators

- & (address operator)
  - Returns address of operand

```
int y = 5;
int *yPtr;
yPtr = &y;                    /* yPtr gets address of y */
yPtr "points to" y
```



Pointer Operators

- * (indirection/dereferencing operator)
  - Returns a synonym/alias of what its operand points to
  - `*yptr` returns y (because `yptr` points to `y`)
  - * can be used for assignment
    - Returns alias to an object

```
*yptr = 7;          /* changes y to 7 */
```

  - Dereferenced pointer (operand of *) must be a value (no constants)
- *** and & are inverses**
  - They cancel each other

Pointer Operators

- The indirection operator, * , can be considered as the complement to the address operator. It **returns the contents of the address stored in a pointer variable**. It is used as follows:

```
*p;
```

- The above expression is read as "contents of p". What is returned is the value stored at the memory address p.
- Consider the sample code:

```
#include <stdio.h>
main() {
    int a=1,b=78,*ip;
    ip=&a;
    b=*ip;      /* equivalent to b=a */
    printf("The value of b is %d\n",b); }
```

```
The value of b is 1
```

- Note that **b** ends up with the value of **a** but it is done **indirectly**; by using a pointer to **a**.

```c
/* Pointer example */
#include <stdio.h>

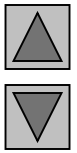
int main(void){
    int m=25;
    int *p;

    p = &m;  /* Recall the address operator & */

    printf("*p = %d\n", *p);
    printf("p = %d\n", p);

    return(0);
}
```

*p (25)is the value stored at the specified address
p (10FE234567) is the corresponding address information



Outline

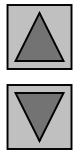
```
3  #include <stdio.h>
4
5  int main()
6  {
7     int a;        /* a is an integer */
8     int *aPtr;    /* aPtr is a pointer to an integer */
9
10    a = 7;
11    aPtr = &a;    /* aPtr set to address of a */
12
13    printf( "The address of a is %p"
14            "\nThe value of aPtr is %p", &a, aPtr );
15
16    printf( "\n\nThe value of a is %d"
17            "\nThe value of *aPtr is %d", a, *aPtr );
18
19    printf( "\n\nShowing that * and & are complements of "
20            "each other\n&*aPtr = %p"
21            "\n*&aPtr = %p\n", &*aPtr, *&aPtr );
22
23    return 0; /* indicates successful termination */
24
25 } /* end main */
```

The address of a is the value of aPtr.

The * operator returns an alias to what its operand points to. aPtr points to a, so *aPtr returns a.

Notice how * and & are inverses

© Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved.



Outline

Program Output

```
The address of a is 0012FF7C
The value of aPtr is 0012FF7C

The value of a is 7
The value of *aPtr is 7

Showing that * and & are complements of each
other.
&*aPtr = 0012FF7C
*&aPtr = 0012FF7C
```

© Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved.

Indirect Reference

Unary operator * is called the indirection operator

```c
/* Pointer example */
#include <stdio.h>

int main(void){
    int m=25;
    int *p;

    p = &m;  /* Address of m is stored in p */

    *p = 40; /* The value stored in p is updated */

    printf("m becomes %d\n",m);

    return(0);
}
```

Note that although we did not update m directly, its value became 40

```c
/* Functions with output parameters */
#include <stdio.h>

void update (int x); /* Function prototype */

int main(void){
    int x = 5;

    update(x); /* Attempt to update x */

    printf("The value of x is %d\n",x);
    return (0);
}

/* Tries to update */
void update (int x){
    x++;
}
```

The value of x is 5

```c
/* Functions with output parameters */
#include <stdio.h>

void update (int *x); /* Function prototype */

int main(void){
    int x = 5;

    update(&x); /* Attempt to update x */

    printf("The value of x is %d\n",x);
    return (0);
}

/* Tries to update */
void update (int *x){
    (*x)++;
}
```

The value of x is 6

```c
/* Functions with output parameters */
#include <stdio.h>

void update (int x); /* Function prototyp

int main(void){
    int x = 5;

    update(x); /* Attempt to update x */

    printf("The value of x is %d\n",x);
    return (0);
}

/* Tries to update */
void update (int x){
    x++;
}
```

The value of x is 5

The value of x is passed to function. Since the function did not receive the address of x, its value remained same after the function call.

```c
/* Functions with output parameters */
#include <stdio.h>

void update (int *x); /* Function prototyp

int main(void){
    int x = 5;

    update(&x); /* Attempt to update x */

    printf("The value of x is %d\n",x);
    return (0);
}

/* Tries to update */
void update (int *x){
    (*x)++;
}
```

The value of x is 6

The address of x in the memory is passed to function